ÉCOLE MAROCAINE DES SCIENCES DE L'INGÉNIEUR (EMSI)
Module : Développement et déploiement d'applications intelligentes - Année universitaire 2026-2027
Encadrante : Pr. Soukaina Gouraguine

Compte rendu

Atelier 3 : API REST avec FastAPI
Le projet LogiAI

Prédire le risque de retard d'une livraison et l'exposer par une API

Étudiants	Taha El Azzouzi, Sara Boufares
Groupe	G4
Dépôt Git	github.com/taha-elazzouzi/logiai-tp (branche tp-api)

1. Environnement et premiers tests

Après l'installation de l'environnement virtuel, la génération des données (5000 livraisons, 27,4 % de retards) et l'entraînement du modèle, nous avons lancé la suite de tests avant d'écrire la moindre ligne de code :

```
collected 13 items
tests\test_api.py ..FFFF......F
FAILED test_predict_valide_renvoie_200 - assert 422 == 200
FAILED test_predict_forme_de_la_reponse - assert {'detail'} == {...}
FAILED test_predict_probabilite_entre_0_et_1 - KeyError
FAILED test_predict_trajet_risque_plus_eleve_que_trajet_facile - KeyError
FAILED test_modele_absent_renvoie_503 - assert 422 == 503
5 failed, 8 passed
```

Résultats de l'entraînement (`python -m logiai.train`) :

Mesure	Valeur
Accuracy de la baseline (toujours « à l'heure »)	0,726
Accuracy du modèle	0,774
Rappel	0,464
Précision	0,617
AUC	0,769

Le modèle ne gagne que 4,8 points d'accuracy sur la baseline, car 72,6 % des livraisons sont déjà à l'heure : l'accuracy seule est trompeuse. Le rappel de 0,464 signifie que, sur 100 livraisons réellement en retard, le modèle en détecte environ 46. Même avec un rappel faible, l'API reste utile : un dispatcher prévenu à temps peut réaffecter un véhicule, prévenir le client ou modifier l'itinéraire pour près de la moitié des retards, alors que sans modèle il n'en détecte aucun.

Pourquoi des tests qui attendent un 422 passent-ils déjà ?

Au départ, le schéma `Livraison` ne contient qu'un seul champ (`distance_km`) et il est configuré avec `extra="forbid"`. Toute requête qui contient les sept champs du contrat est donc refusée avec un code 422 à cause des six champs « inconnus », **quelle que soit la valeur envoyée**. Ces tests sont verts pour la mauvaise raison : ils ne vérifient pas que la pluie de 500 mm ou l'« avion » sont refusés, ils constatent seulement que tout est refusé. Le test de succès (code 200) les démasque : il échoue, ce qui révèle que le schéma est incomplet. Un test vert ne suffit donc pas : il faut savoir pourquoi il passe.

2. Schéma de validation

Les bornes sont des bornes physiques (ce qu'un trajet réel peut afficher), et non le minimum et le maximum observés dans `livraisons.csv`, afin de ne pas refuser une livraison réelle légèrement plus longue ou plus lourde.

Variable	Type	Min	Max	Justification
distance_km	float	1	1000	Valeur donnée par l'énoncé. Un trajet de moins de 1 km n'a pas de sens ; 1000 km couvre un trajet national en une journée.
poids_kg	float	0	40000	Un poids n'est jamais négatif. 40 tonnes est la charge maximale d'un poids lourd routier.
heure_depart	int	0	23	Heure du jour, entière : 24 n'existe pas (minuit vaut 0).
pluie_mm	float	0	100	Pas de pluie négative. 100 mm est un cumul extrême ; 500 mm est physiquement impossible et doit être refusé.
trafic_index	float	0	10	Échelle de l'énoncé : 0 = fluide, 10 = saturé.
nb_arrets	int	0	50	Entier positif ou nul ; 50 arrêts dépasse déjà largement une tournée réaliste.
type_vehicule	Literal	-	-	moto, fourgon ou camion : tout autre texte (par exemple « avion ») est refusé.

Le schéma est aussi configuré avec `extra="forbid"` : un champ inconnu est refusé avec un code 422. Il est écrit une seule fois et FastAPI s'en sert pour valider les requêtes, documenter la page `/docs` et fournir un objet Python typé à la fonction.

3. La fonction predict()

La fonction récupère le prédicteur dans l'état de l'application, répond 503 s'il est absent, sinon renvoie la prédiction typée.

```
@app.post("/v1/predict", response_model=PredictionOut)
def predict(livraison: Livraison):
    predictor = state.get("predictor")
    if predictor is None:
        raise HTTPException(
            status_code=503,
            detail="Modèle non chargé : lancez "
                   "'python -m logiai.train' puis relancez le serveur.",
        )
    return PredictionOut(**predictor.predict(livraison.model_dump()))
```

4. Quatre requêtes

Requête	Code obtenu	Pourquoi
Livraison normale	200	Les sept champs sont valides et le modèle est chargé : l'API renvoie la probabilité, l'alerte, le seuil et la version.
distance_km = -5	422	Le schéma refuse la valeur (ge=1) avant même d'appeler predict() : le modèle n'est jamais appelé.
type_vehicule = "avion"	422	La valeur n'appartient pas au Literal ["moto", "fourgon", "camion"].
Modèle absent (dossier models/ supprimé)	503	Le serveur démarre mais l'état ne contient pas de prédicteur : predict() répond « service indisponible ». /health répond 200 avec model_loaded: false.

5. Questions de compréhension

1. Pourquoi charge-t-on le modèle au démarrage et pas à chaque requête ?

Lire `model.joblib` sur le disque et le désérialiser est lent. Le faire à chaque requête ajouterait ce délai à chaque réponse et saturerait le disque et la mémoire dès que plusieurs requêtes arrivent ensemble. Chargé une seule fois dans `lifespan`, le modèle reste en mémoire et chaque requête n'a plus qu'à calculer la prédiction.

2. Qui écrit le contrôle « pluie <= 100 », et où est-il écrit ?

C'est le développeur, dans le schéma Pydantic `Livraison` (fichier `logiai/api/schemas.py`), avec `Field(ge=0, le=100)`. FastAPI exécute ce contrôle automatiquement avant d'appeler `predict()`.

3. Différence entre 422 et 503 ?

Le 422 est une erreur du **client** (famille 4xx) : la requête est mal formée ou contient des valeurs impossibles, le client doit la corriger. Le 503 est une erreur du **serveur** (famille 5xx) : la requête est correcte mais le service ne peut pas la traiter (ici, le modèle n'est pas chargé) ; le client peut réessayer plus tard.

4. Pourquoi une route /v1/predict plutôt que /predict ?

Le préfixe `/v1` est la version de l'API. Quand le contrat change (nouveau champ, nouvelle forme de réponse), on peut publier `/v2/predict` sans casser les clients existants (Streamlit, application mobile) qui continuent d'appeler `/v1`.

5. Que se passerait-il si la distance était déclarée int ?

Les distances décimales (par exemple 12,5 km) seraient refusées avec un code 422 (« Input should be a valid integer »), alors qu'une distance est naturellement un nombre à virgule. Les clients seraient obligés d'arrondir, ce qui fausserait la prédiction.

6. Décision : modèle absent

Nous choisissons **l'option A : démarrer quand même et répondre 503** aux demandes de prédiction.

Ce que fait perdre l'option A. Tant que le modèle manque, aucune prédiction n'est possible. De plus, un serveur « vivant » mais inutile peut passer inaperçu, et un client peut croire que le service fonctionne.

Ce que fait perdre l'option B (refuser de démarrer). Tout le service s'arrête, y compris `/health` et `/docs`. La personne qui supervise le service ne peut plus interroger `/health` pour comprendre ce qui se passe : elle voit seulement un service qui ne répond pas, sans en connaître la cause.

Justification. Avec l'option A, `/health` répond 200 avec `model_loaded: false` et `/v1/predict` répond 503 avec un message qui indique quoi faire. L'erreur est donc visible et diagnostiquable, et les clients reçoivent un code HTTP clair (503 signifie « service temporairement indisponible ») plutôt qu'un silence. En production, on complète par une alerte sur `model_loaded: false`.

7. Lien de la pull request

Dépôt : https://github.com/taha-elazzouzi/logiai-tp (branche `tp-api`).

Pull request : .....